

Problem Definition

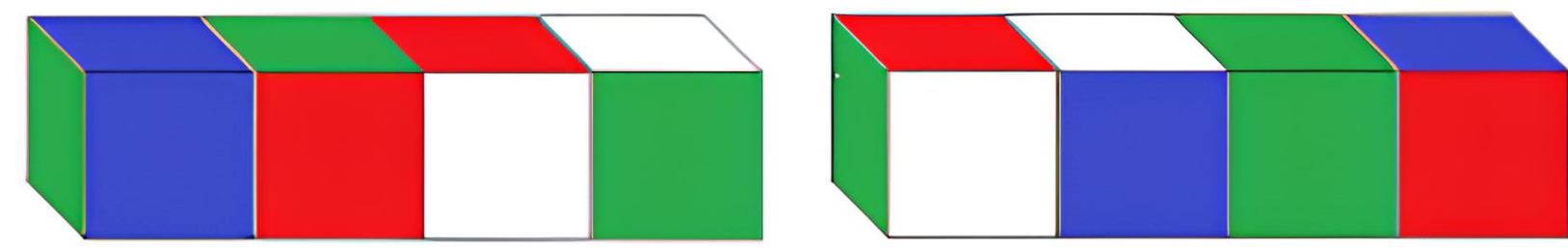


Figure 1. Problem definition of Instant Insanity: Example solution

Given a set of blocks, each with different configurations, the goal of Instant Insanity is to stack the four cubes so that each color appears exactly once on each of the four sides of the tower.

Mapping the Cube to a Graph and Encoding

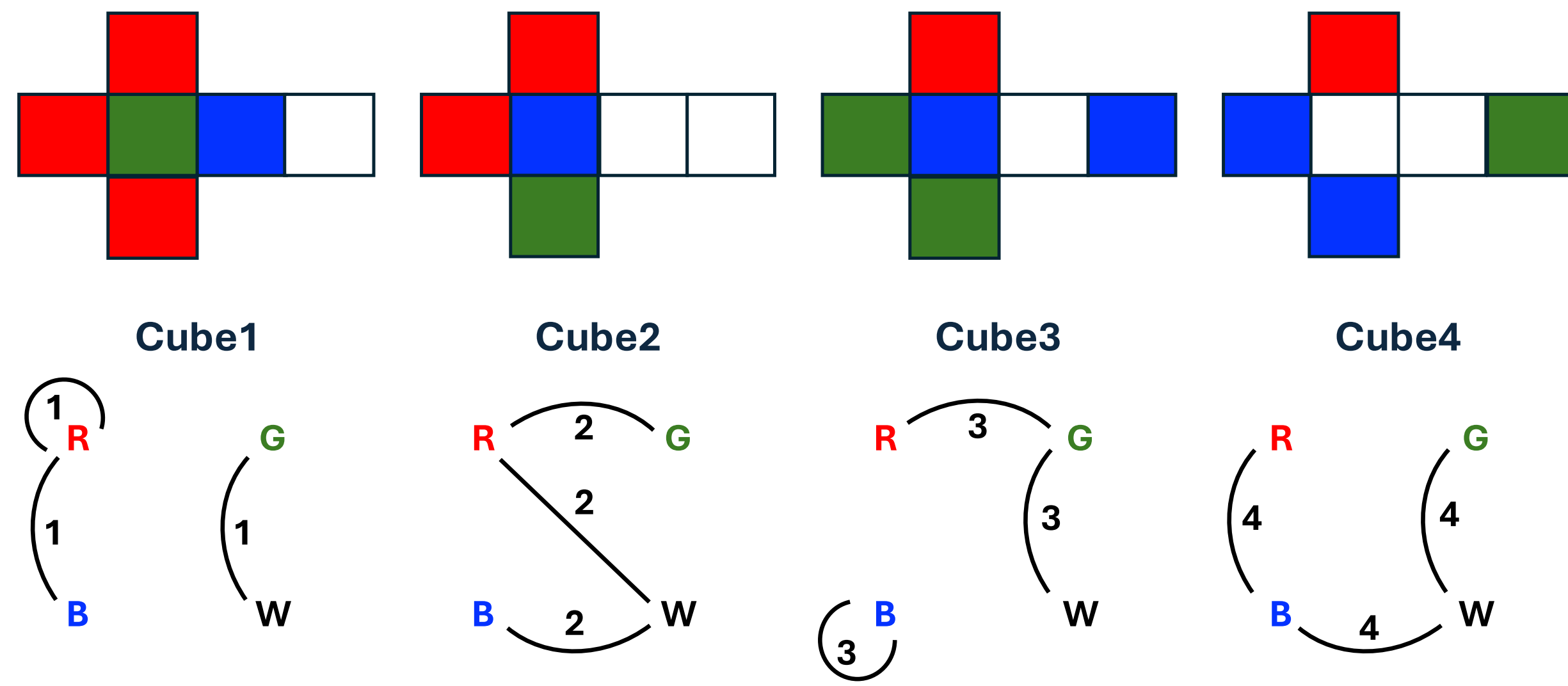


Figure 2. Graph representation of each cube and combined graph representation of all cubes.

Each cube is represented by three edges, each corresponding to a pair of opposite faces. For each edge, the two possible directions-top/bottom and front/back-are regarded as distinct cases.

Graph Encoding

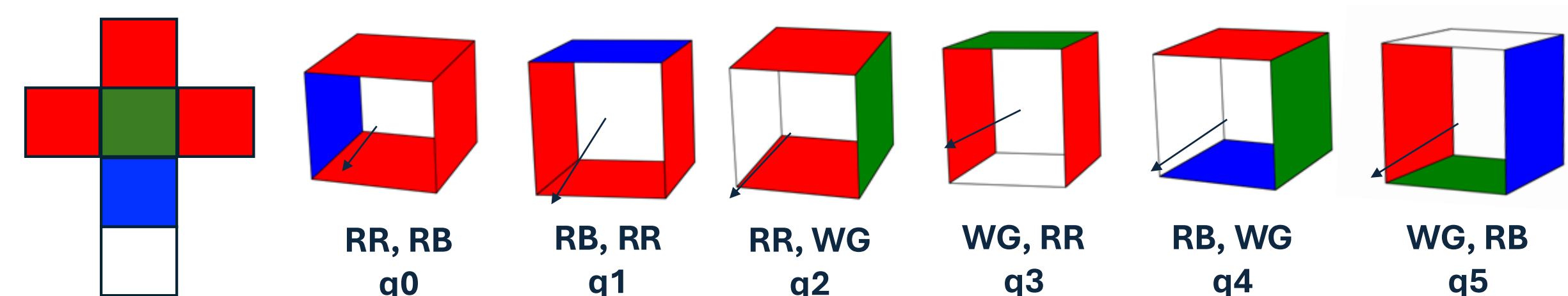


Figure 3. Six possible states per cube, considering configuration and direction.

Cube	1			2						3						4					
Qubit	q ₀	q ₁	q ₂	q ₃	q ₄	q ₅	q ₆	q ₇	q ₈	q ₉	q ₁₀	q ₁₁	q ₁₂	q ₁₃	q ₁₄	q ₁₅	q ₁₆	q ₁₇	q ₁₈	q ₁₉	q ₂₀
A	RR	RR	RB	RW	BW	RG	BW	RG	RW	GW	BB	RG	BB	RG	GW	BR	WG	BW	WG	BW	BR
B	RB	WG	WG	BW	RW	BW	RG	RW	RG	BB	GW	BB	RG	GW	RG	WG	BR	WG	BW	BR	BW

Figure 4. Encoding of all cube states onto 21 qubits.

Each qubit represents the state of a cube. Each cube has six possible states, as shown in Fig. 3, depending on which pair of faces are assigned as top/bottom and front/back. The first cube requires only three qubits, since the distinction between top/bottom and front/back is relative. Therefore, a total of 21 qubits (3 + 6 + 6 + 6) are needed, as shown in Fig. 4.

Classical Divide-and-Conquer Approach

The graph encoding in the previous section attempts to solve the problem by incorporating both the top/bottom and front/back configurations into the cost function. However, due to its complexity, this approach took over an hour to run and produced results with lower accuracy. To address this, we adopted an alternative scheme that considers only a single pair of rows of faces (either top/bottom or front/back). This approach allows us to narrow down the candidate solutions and select one with zero cost. We then run the next trial to derive new candidates for the remaining pair of rows, using an updated cost function that reflects the first solution.

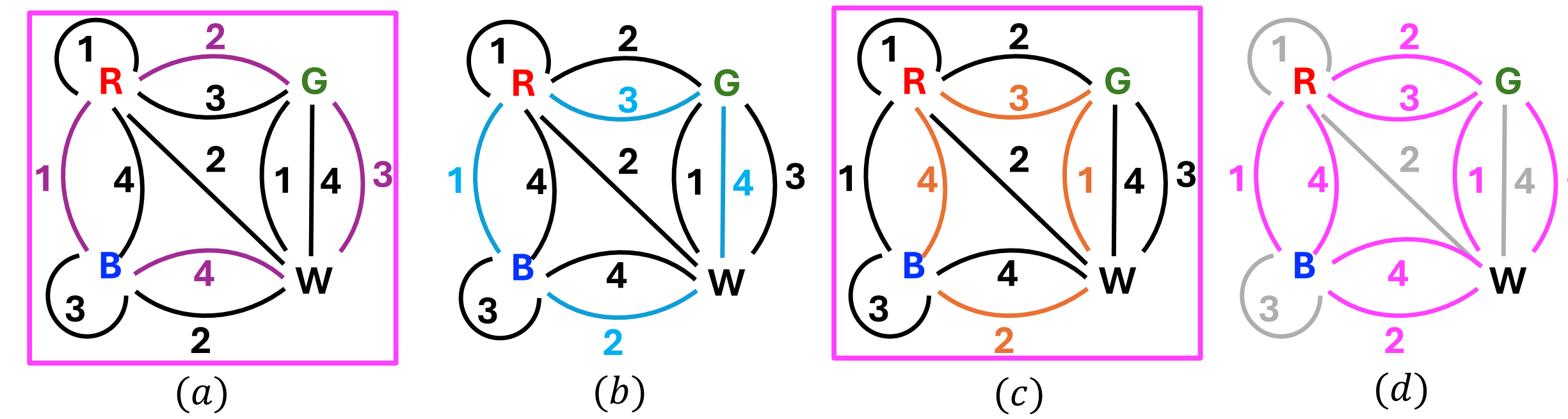


Figure 5. (a)–(c): Possible solutions for one pair of rows; (d): Final solution obtained by combining (a) and (c).

To satisfy the condition that each color appears exactly once on each rows, every vertex (color) must have degree two (appear twice), which implies that a cycle must be formed in the combined graph of all cubes. Fig. 5 (a)–(c) show possible candidates for one pair of rows. The final solution (d) is then obtained by combining two partial candidates (a) and (c) that do not share edges.

Cube	1 (x)			2 (y)			3 (z)			4 (w)		
Qubit	q ₀ (x ₁)	q ₁ (x ₂)	q ₂ (x ₃)	q ₃ (y ₁)	q ₄ (y ₂)	q ₅ (y ₃)	q ₆ (z ₁)	q ₇ (z ₂)	q ₈ (z ₃)	q ₉ (w ₁)	q ₁₀ (w ₂)	q ₁₁ (w ₃)
Pair	RR	RB	WG	RG	RW	BW	RG	GW	BB	BR	WG	BW

Figure 6. Encoding of all states onto 12 qubits, with each edge as a qubit per cube.

Each qubit represents the configuration of a pair of opposite faces, corresponding to one edge of the cube in Fig. 2. Thus, each cube can be in one of three possible states, and a total of 12 qubits (3 + 3 + 3 + 3) are required, as illustrated in Fig. 6.

QUBO and Hamiltonian from Graph Constraints

- Constraint 1:** Each cube should occupy exactly one state. (one-hot per cube)

$$C_{cube} = (x_1 + x_2 + x_3 - 1)^2 + (y_1 + y_2 + y_3 - 1)^2 + (z_1 + z_2 + z_3 - 1)^2 + (w_1 + w_2 + w_3 - 1)^2$$
- Constraint 2:** From the graph, every color should appear twice within one pair of rows. Therefore, there are four terms, one for each color: $C_{graph} = C_r + C_g + C_b + C_w$.

$$C_{graph} = (2x_1 + x_2 + y_1 + y_2 + z_1 + w_1 - 2)^2 + (x_3 + y_1 + z_1 + z_2 + w_2 - 2)^2 + (x_2 + y_3 + 2z_3 + w_1 + w_3 - 2)^2 + (x_3 + y_2 + y_3 + z_2 + w_2 + w_3 - 2)^2$$

We can set the QUBO as

$$QUBO = C_{cube} + C_{graph}$$

However, to reduce the size of the QUBO (and consequently the Hamiltonian), we can satisfy the first constraint C_{cube} by preparing the initial state as a tensor product of Dicke states for each cube. The Hamiltonian can be derived by substituting $var \in \{0, 1\}$ in the QUBO expression and rewriting the cost function in terms of $z_i \in \{-1, 1\}$, resulting in an Ising model. We employed the QAOA Ansatz, Estimator, and Optimizer in the same manner as in the Four Corners Map Coloring.

Dicke State Preparation and Associated XY-Mixers

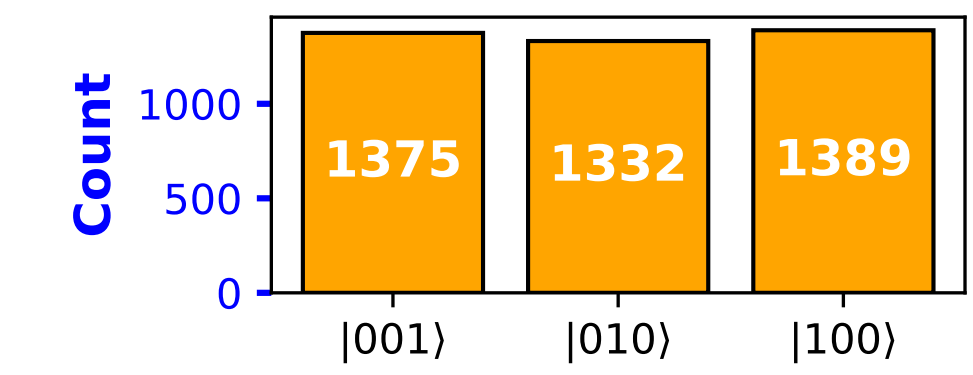


Figure 7. Dicke-state preparation for the initial state of each cube.

$$U_{\text{Dicke-mixer}}(\beta) = \exp\left(-i\beta \sum_{(u,v) \in \text{pairs of cube edges}} (X_u X_v + Y_u Y_v)\right)$$

As we prepared the tensor product of Dicke states as the initial state, we implement a special mixer that preserves the Dicke subspace $\{|001\rangle, |010\rangle, |100\rangle\}$. In this way, the constraints are preserved: $|001\rangle$, $|010\rangle$, and $|100\rangle$ rotate (mix) by β through the mixer.

First Iteration and Post-Processing

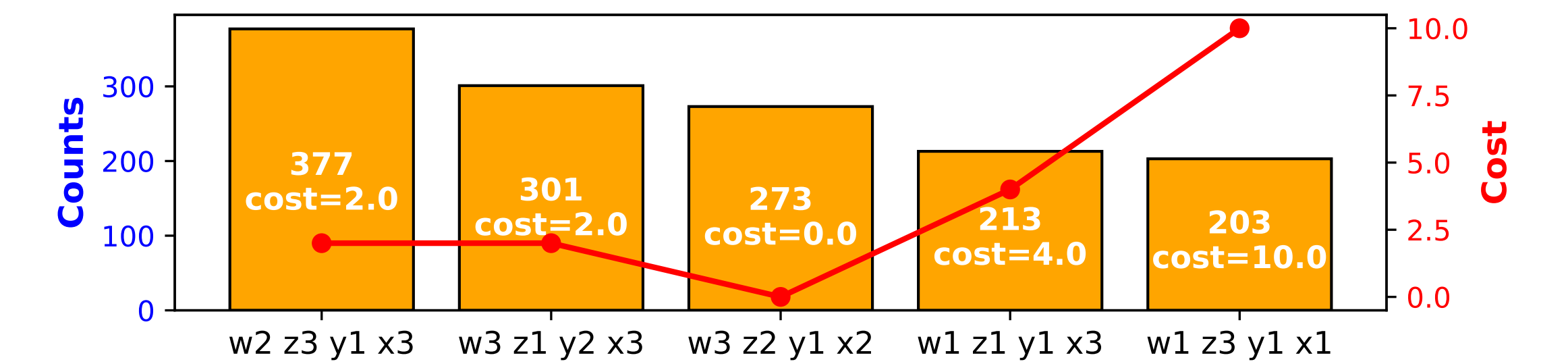


Figure 8. Top-5 candidates for the first pair of rows; the zero-cost solution $\{w_3, z_3, y_1, x_3\}$ is selected.

Based on the result of the first iteration for one pair of rows, we suggest two approaches for post-processing in the next iteration for the remaining pair.

- Approach 1:** Add the bias term of the first solution with zero cost $\{w_3, z_3, y_1, x_3\}$. In this approach, we apply a penalty strength of 5 and set the QAOA circuit depth to $p = 15$.

$$C_{bias} = 5(x_2 + y_1 + z_2 + w_3)^2, C_{graph} \text{ (2nd iteration)} = C_{graph} \text{ (1st iteration)} + C_{bias}$$
- Approach 2:** Delete the solution variables $\{w_3, z_3, y_1, x_3\}$ from the previous C_{graph} , since edges should not be reused. Another solution will then be selected from the remaining edges.

Results

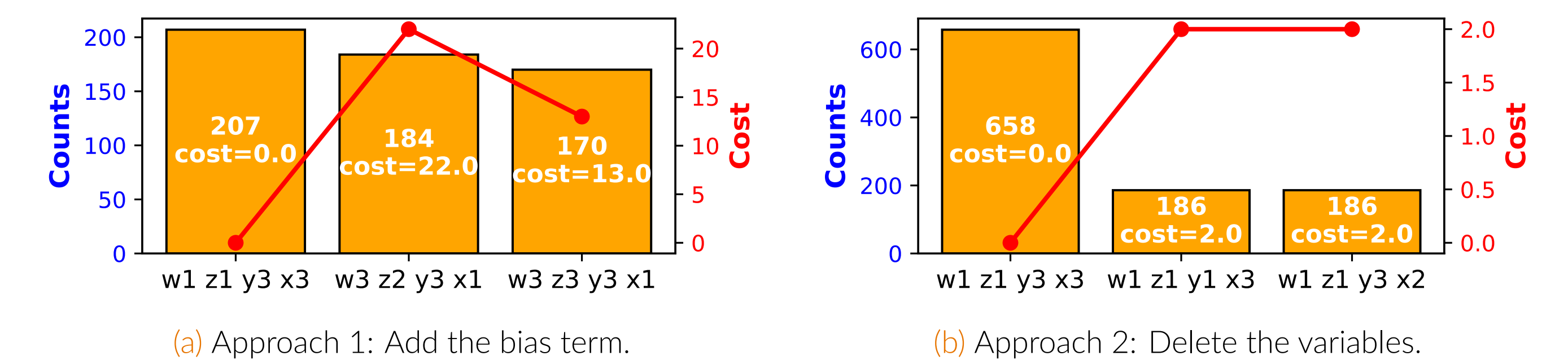


Figure 9. Top-3 candidates for the remaining under each approach; the zero-cost solution $\{w_1, z_1, y_3, x_3\}$ is selected.

For the final check, with the first solution: Fig.5(a) and the second solution: Fig.5(c), we verified that no edges are shared. Therefore, the correct solution for the whole cube is obtained, which corresponds to Fig.5(d). Note that there is no corresponding zero-cost solution for Fig.5(b).